BIRZEIT UNIVERSITY

Department of Electrical and Computer Engineering
Advanced Distributed Systems- ENCS5324

Assignment#1 due 21/11/2026

Assignment: Distributed Proof-of-Work Nonce Search with REST and gRPC

Part 1: Concepts (written answers)

1. **Synchronous vs. asynchronous RPC.** In your own words, explain the difference between synchronous and asynchronous RPC calls. Give one situation where each is the better choice.
2. **gRPC communication patterns.** Explain the four gRPC method types (unary, server streaming, client streaming, and bidirectional streaming) and give a realistic example of each.
3. **REST vs. gRPC.** Compare REST and gRPC, listing at least **three similarities** and **three differences**. Explain why many systems use REST for external clients and gRPC for communication between their internal components.
4. **Long-running operations over REST.** A proof-of-work search can take minutes, so an HTTP request can't simply wait for the answer. Explain the *asynchronous job* pattern (submit a job, receive a job ID, then poll for or stream its status). Also explain how **Server-Sent Events (SSE)** let a server push updates to a client over HTTP.

Part 2: A Streaming RPC Service

Design and implement a small gRPC service in **Go** that uses **unary and streaming method**. Examples include a chat room, a live stock-price feed, a file uploader that sends a file in chunks, or a log viewer. Requirements:

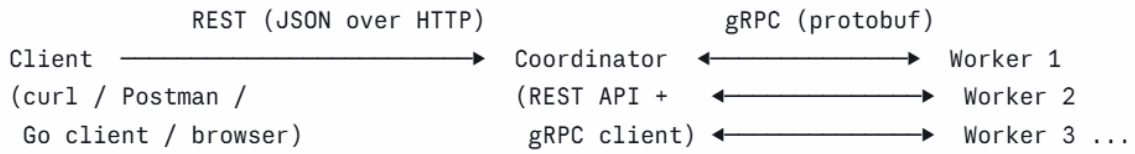
- Define the service and messages in a **.proto file**.
- Include **at least one unary method** and **at least one streaming method**.
- Show runs that demonstrate each method working, including messages arriving over the stream.

Part 3: Distributed Proof-of-Work Nonce Search

Using Go, gRPC, Protocol Buffers, and a REST API, build a distributed system that solves the following puzzle:

Given a string data and a difficulty d, find a non-negative integer nonce such that the SHA-256 hash of data + nonce (in hexadecimal) starts with d zeros.

Architecture



1. Coordinator: REST API for clients

The coordinator exposes the following JSON endpoints:

HTTP method	Endpoint	Description
POST	/jobs	Submit a job with a body like {"data": "hello", "difficulty": 5}. Returns **202 Accepted** and {"job_id": "..."}.
GET	/jobs/{id}	Return the job's status (RUNNING, FOUND, or CANCELLED), the number of nonces checked, the elapsed time, and the nonce and hash once found.
GET	/jobs/{id}/events	Stream live progress updates to the client with **Server-Sent Events**.
DELETE	/jobs/{id}	Cancel a running job. All of the job's worker searches must stop.
GET	/jobs	List all jobs and their statuses.
GET	/workers	List the registered workers, whether each one is alive, and its current assignment.

Requirements for the REST API:

- Validate input. For example, a missing data field or a difficulty outside the range 1–8 should return **400 Bad Request**.
- An unknown job ID should return **404 Not Found**.
- Cancelling a job that has already finished should return **409 Conflict**.
- The coordinator must be able to handle **several jobs at the same time**.

2. Coordinator ↔ workers: gRPC

- Workers call RegisterWorker when they start and send a Heartbeat periodically.
- The coordinator splits the nonce search space into sub-ranges, such as $[0, 10^6)$, $[10^6, 2 \cdot 10^6)$, and so on. It assigns each range to a worker with a Search RPC.
- Workers report progress through a **streaming RPC**: either Search is server-streaming and returns progress messages followed by the result, or workers use a separate client-streaming ReportProgress method.
- When a worker finishes a range without finding a solution, the coordinator assigns it the next unassigned range.

3. Workers

- Each worker searches its assigned range and returns either a valid nonce or a message saying its range contains no solution.
- Each worker must stop promptly when its search's context is cancelled.

Required behavior

4. **Early termination:** as soon as any worker finds a valid nonce, the coordinator must **cancel all other workers' searches for that job**, mark the job as FOUND, and send a final event to SSE subscribers.
5. **Client-initiated cancellation:** a DELETE /jobs/{id} request must propagate through gRPC context cancellation to every worker searching for that job.
6. **Fault tolerance:** if a worker stops sending heartbeats or a Search call fails, the coordinator must reassign the worker's range to another worker.
7. The number of workers, the size of each range, and the coordinator's HTTP and gRPC ports must be configurable, for example through command-line flags.

Part 4: Testing and Experiments

1. **REST workflow:** use curl or Postman to submit a job, poll its status, watch the SSE stream (for example with curl -N http://localhost:8080/jobs/{id}/events), and retrieve the final result.
2. **Difficulty:** run jobs with **d = 4, 5, and 6**. For each one, record the nonce, the hash, and the execution time.
3. **Scalability:** repeat the d = 5 test with **1, 2, 4, and 8 workers**. Present the times in a table or chart and discuss the speedup.
4. **Concurrent jobs:** submit **3 jobs at the same time** and show that all of them complete correctly.
5. **Cancellation:** cancel a running job with DELETE and show in the worker logs that its searches stopped.
6. **Fault tolerance:** stop a worker in the middle of a search and show that its range was reassigned and the job still finished.
7. **Error handling:** show the HTTP status codes returned for invalid input, an unknown job ID, and cancelling a job that has already finished.
8. **Bonus (optional):** build a simple HTML page, served by the coordinator, that submits jobs and shows live progress using EventSource.

Deliverables

Submit a single report (PDF) that includes:

- Your answers to Part 1.
- An architecture diagram and a short description of your design, covering the REST API, the gRPC services, and how cancellation travels from an HTTP request down to the workers.
- The .proto files and all Go source code, either in the report or as an attached archive or repository link.
- Instructions for building and running the coordinator, the workers, and the client.
- **Screenshots** of every test, showing the curl or Postman output, the SSE stream, and the coordinator and worker logs.
- The scalability table or chart and a brief discussion of the results.

You can start learning go at :
https://go.dev/doc/tutorial/getting-started